Hackathon Judging Guide

Welcome, and Thank You!

Whether you've judged a dozen hackathons or this is your very first one, this guide is here to make the experience straightforward and enjoyable. You don't need to be a software engineer or a technical expert to be a great judge. Hackathons value creativity, ambition, and learning just as much as raw technical skill, and your perspective matters.

So, What Are You Actually Judging?

A hackathon is a time-limited event where participants, solo or in teams, build a project largely from scratch in 24 hours. The result is rarely a polished, production-ready product, and that's completely fine. You're not evaluating whether something is ready to ship. You're evaluating the idea, the effort, the learning, and what they managed to pull together under real pressure.

Try to approach each project with a spirit of curiosity. Ask yourself (and the participants, if you are unsure!): **What were they trying to do, and how well did they do it given the constraints they were working under?**

Scoring

You'll score each project across up to six categories, each on a scale of **1 to 5**. A 5 means you were genuinely impressed. A 1 means the team missed the mark significantly. Most scores will naturally land in the middle.

Don't stress about being mathematically precise, go with your honest gut read for each category. You don't need to know how the scores are weighted or combined; our system takes care of all of that.

MOST IMPORTANTLY: Be consistent. I'll stress this again later because it REALLY matters. You don't need to give scores that you think organizers want, or scores that other judges are giving. Grade projects on a system that is internally consistent to other projects you grade.

The Categories

Creativity *Up to 5 points*

Did this project surprise you? Creativity isn't just about wild ideas, it's about originality and the willingness to approach a problem differently. A creative project might tackle a familiar problem in an unexpected way, combine concepts you wouldn't normally see together, or build something you genuinely hadn't thought of before.

A 5 here means you walked away thinking **"I never would have thought of that."** A 1 means the project felt generic or derivative, something you've seen many times before with no meaningful twist.

> **Example [5]:** Spiral transforms C and Python source code into an explorable 3D city rendered in your browser. Every function call, loop, branch, and variable assignment becomes a building you can fly through and click on. Turning a debugger into an art piece is exactly the kind of idea that makes you say *"I never would have thought of that."*

> **Example [1-2]:** Goosepedia is a clean, well-structured website covering goose anatomy, diet, and behavior. The execution is solid, but the concept is a simple informational site with no unexpected angle or twist.

Most Learned *Up to 5 points*

Hackathons are fundamentally about growth. This category rewards teams who pushed themselves, who tried something new, took on a challenge outside their comfort zone, or tackled a hard problem even if they didn't fully solve it. During presentations, listen for moments where teams talk about what they struggled with, what they figured out, and what they'd do differently. (Maybe even ask for these if a team is not clearly communicating!) A team that built something simple but learned a ton can absolutely outscore a team that played it safe with tools they already knew.

A 5 here means the team clearly stretched themselves and can speak meaningfully about what they gained. A 1 means there's little evidence of growth or challenge taken on.

> **Example [5]:** EduSynapse implemented a custom AI model training approach straight from a recent academic paper, wrote their own spaced repetition algorithm from scratch, and integrated a locally hosted AI they had no prior experience with.

> **Example [2-4]:** Snap&Serve had to abandon React Native mid-hackathon and rebuild in React after discovering camera support limitations they hadn't anticipated. Pivoting under pressure and shipping anyway is a real lesson, though it was a gap in upfront research rather than a deliberate stretch.

Technicality *Up to 5 points*

This one is about execution, how well was the project actually built? For non-technical judges, you don't need to evaluate the code. You can assess this through the lens of: **Does it work? Does it seem well thought-out? Did they handle complexity well?** For more technical judges, feel free to dig into architecture, implementation choices, or how they handled edge cases.

Importantly, try to feel out how well the participants seem like they know what they are talking about. If you ask how something works, and they stumble through an explanation, they most likely do not understand it fully. If they can explain it, but only with extremely complex terminology, that still might not mean they understand it, but rather that they memorized something. If they can communicate it in a way that YOU understand, regardless of technical ability, then you have a technical project.

A 5 means the technical execution was impressive and clearly required significant skill or effort. A 1 means the project was barely functional or technically very thin.

> **Example [5]:** narr0w is a full AI-powered ticket triage pipeline. It evaluates incoming tickets against each team member's skills and workload, assigns them with a confidence score, and includes a "Code-Mode" agent that autonomously analyzes a repository, breaks the task into steps, creates GitHub branches, and submits pull requests. Every layer is purposeful and clearly understood by the team.

> **Example [1-2]:** Goosepedia is built with static HTML and CSS, no backend, no interactivity, no meaningful technical challenge. That's completely fine for what it set out to be, but it doesn't demonstrate much technical execution.

Note: A technically simple project can still score well here if it's executed cleanly and intentionally. Complexity for its own sake isn't the goal, but rather understanding and proper application of that complexity.

Theme Match ***Up to 5 points (OPTIONAL)***

Many hackathons have a theme, and this category measures how well the project connects to it. Projects DON'T need to follow the theme, it's fully optional.

The theme of this year's KHE is **Futuristic Caveman**. You are free to interpret this as you wish as you judge, and discuss theme fit with participants if you are unsure.

This doesn't mean every project needs to be a literal interpretation, clever or abstract connections to the theme are great. What you're looking for is that the team was clearly aware of the theme and made deliberate choices to align with it.

A 5 means the theme is central to the project you couldn't imagine the project existing without it. A 1 means the team largely ignored the theme or the connection feels like a stretch.

If the team did not attempt the theme at all, you may leave this field blank

(Which is, again, fully okay. Projects are not required to fit the theme.)

Track-Specific Score ***Up to 5 points (OPTIONAL)***

Participants have the option to submit their project to one of our focused challenge tracks. If a team has done so, you'll see their chosen track on their submission, and you should score this category based on how well their project fits and excels within that track's goals.

If a team did not submit to a track, skip this category entirely. It simply won't apply to them.

The tracks for this hackathon are:

Cybersecurity Projects focused on security, encryption, vulnerability detection, penetration testing, or privacy protection. Does the project demonstrate an innovative approach to protecting systems, data, or users from digital threats?

Data Science Projects that use data analysis, machine learning, visualization, or statistical modeling to extract insights or solve real-world problems. Is the data-driven approach meaningful and well-executed?

Embedded Systems Hardware-integrated projects using microcontrollers, sensors, IoT devices, or single-board computers. Does the project meaningfully bridge the digital and physical worlds?

Game Programming Original games or interactive experiences across any genre or platform. Does the project show creativity in gameplay, storytelling, graphics, or player engagement?

For this category, ask yourself: **Given what this track is asking for, how well did this team deliver?**

Overall Score ***Up to 5 points***

Step back and take the project in as a whole. Forget the individual categories for a moment, if you had to summarize your impression of this project in a single score, what would it be? This is your vibe check, and it's allowed to reflect things that don't fit neatly into the other categories

> **Example [5] :**From the moment you see code transformed into a flyable 3D city, you know something special was built. The concept, execution, and polish all point to a team that cared deeply about what they were making.

> **Example [2-3]:** Project Green has an interesting concept (a recycling tracker with a smart trashcan lid using machine learning) and real ambition. Not everything came together in 24 hours, but the vision is clear and the effort shows.

A Few Tips Before You Start

- **Be consistent.** Try to apply the same mental standard across all the projects you see. If you give a 4 for creativity to one team,

ask yourself whether another team's idea was more or less creative before giving them the same score.

- **Take notes.** You may be reviewing several projects in a short window. Jot down a few words after each one so they don't blur together.
- **Be kind, but be honest.** These teams worked hard, but inflating scores doesn't serve anyone. Honest scores make the results meaningful.
- **You don't have to understand everything.** If a team uses a technology or concept you're not familiar with, it's completely fine to

ask them to explain it in plain terms. How well they can do that is itself a useful signal (see both learning and technicality categories!)

Thank you!

Good luck, and thank you for helping make this event great!